Chittagong University of Engineering and Technology



Lab Report

Experiment No. 01: SIMULATION OF SENSORS THROUGH PROTEUS AND ARDUINO.

Course: Biomedical Image Processing (BME 422)

Prepared by:

Name: Rudra Chakraborty

ID: 2011026

Submitted to:

Ayesha Binth Humayun,
Lecturer
Department of Biomedical Engineering,CUET

Submission Date: December 17, 2025

Objectives:

• To learn about sensor connection and simulation in Proteus

• To learn about using Arduino for the connection of Proteus.

Task 1: Solve the limitation of task with LM35 temperature sensor so that in can show temperature greater than 50°C and bellow 0°C temperature appropriately.

Code and output:

```
#include <LiquidCrystal.h>

// LCD wiring
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

// Sensors
#define sensorPin  A0
#define sensorPin1 A1

// LEDs
const int ledGreen = 8;   // Below 0°C
const int ledRed   = 9;   // Above 50°C
const int ledBlue  = 10;  // 0–50°C

const float VREF = 5.0;
const float ADC_MAX = 1023.0;

void setup() {
  analogReference(DEFAULT);

  pinMode(ledGreen, OUTPUT);
  pinMode(ledRed, OUTPUT);
  pinMode(ledBlue, OUTPUT);

  lcd.begin(16, 2);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Temperature:");
}

void loop() {
  int reading0 = analogRead(sensorPin);
  int reading1 = analogRead(sensorPin1);

  float voltage0 = reading0 * (VREF / ADC_MAX);
```

```
  float voltage1 = reading1 * (VREF / ADC_MAX);

  float temperatureC = (voltage0 - voltage1) * 100.0;

  // LCD display
  lcd.setCursor(0, 1);
  lcd.print("                ");
  lcd.setCursor(0, 1);
  lcd.print(temperatureC, 1);
  lcd.print((char)223);
  lcd.print("C");

  // ---- LED LOGIC (UPDATED) ----
  if (temperatureC < 0.0) {
    // Below 0°C → Green
    digitalWrite(ledGreen, HIGH);
    digitalWrite(ledBlue, LOW);
    digitalWrite(ledRed, LOW);
  }
  else if (temperatureC <= 50.0) {
    // 0–50°C → Blue
    digitalWrite(ledGreen, LOW);
    digitalWrite(ledBlue, HIGH);
    digitalWrite(ledRed, LOW);
  }
  else {
    // Above 50°C → Red
    digitalWrite(ledGreen, LOW);
    digitalWrite(ledBlue, LOW);
    digitalWrite(ledRed, HIGH);
  }

  delay(300);
}
```

Output

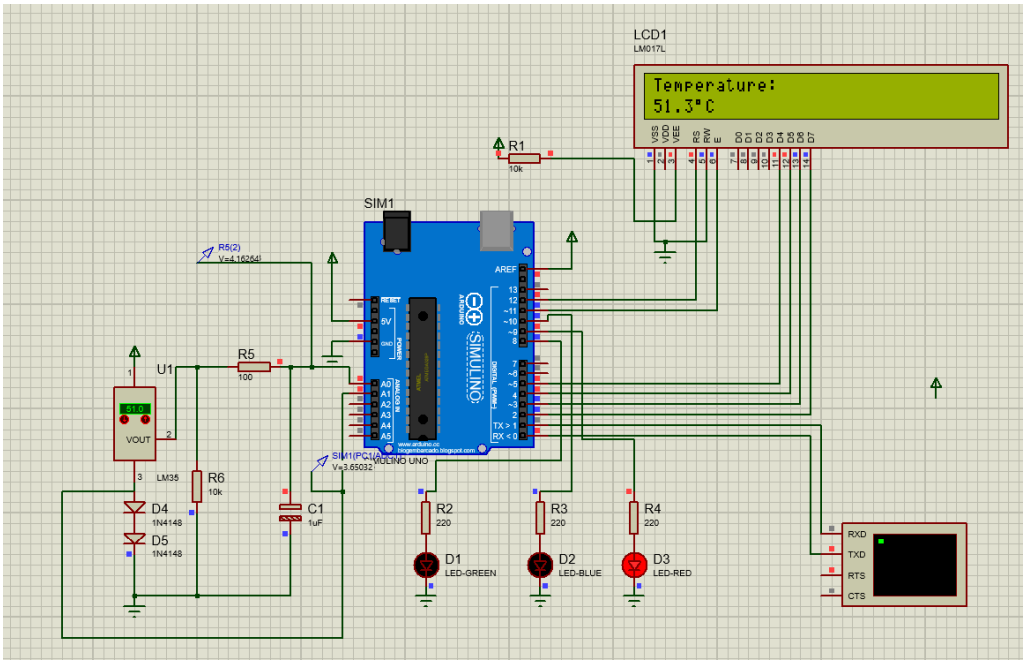


Figure-1

The red LED turns ON when the temperature exceeds 50°C.

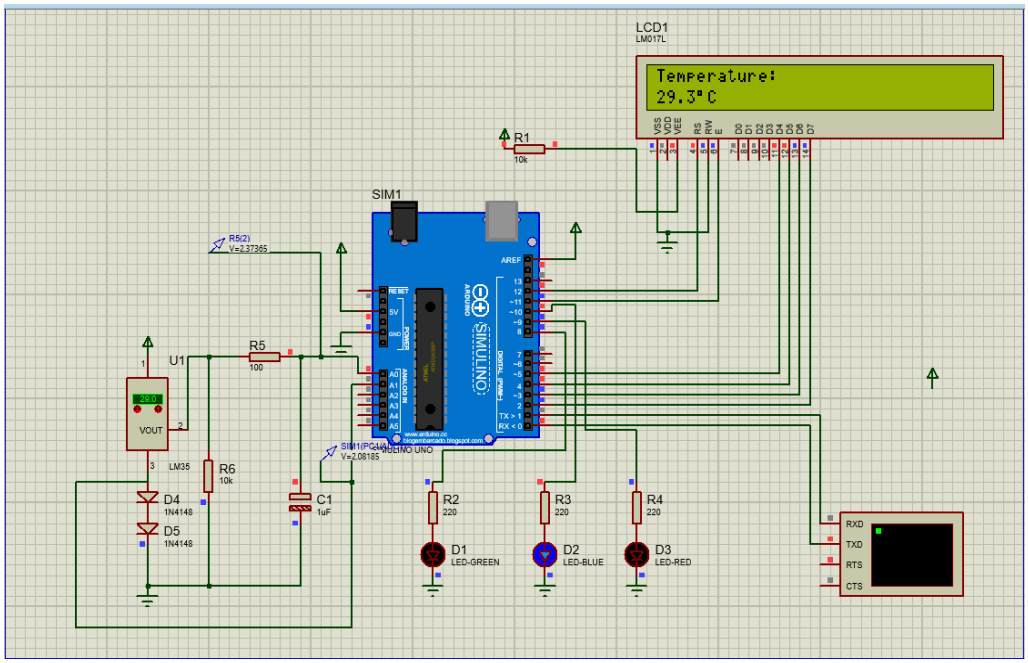


Figure-2

The blue LED turns ON for temperatures between 0°C and 50°C

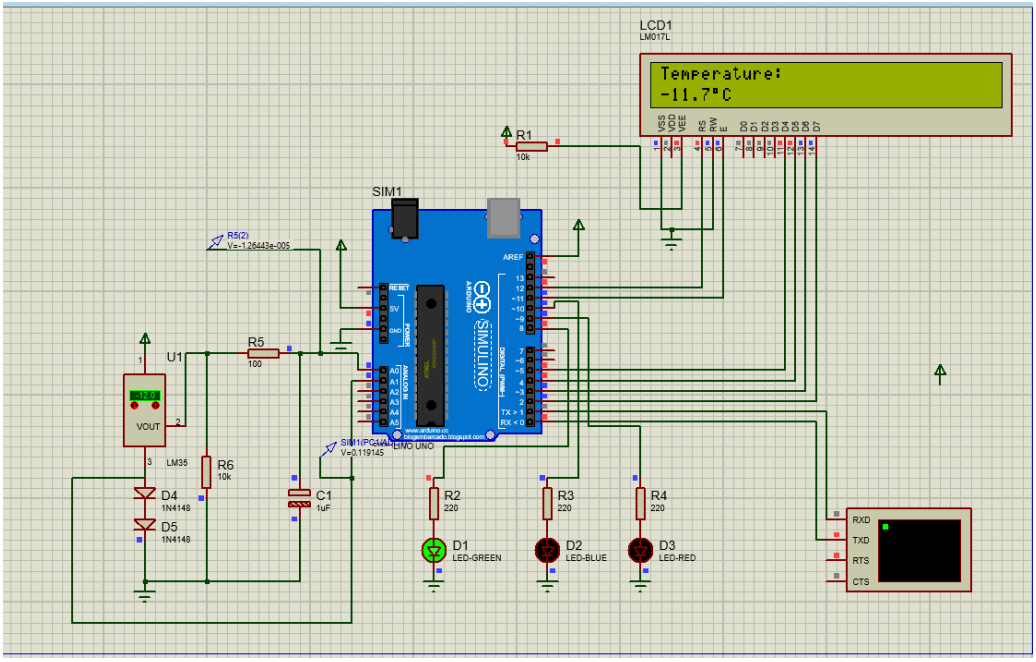


Figure-3

The green LED turns ON for temperatures below 0°C.

Task-2

Connect the ultrasonic sensor, LCD, LED, and buzzer and write a program such that:

1. **Distance > 45 cm**: LCD shows Safe (1st row) and distance (2nd row); LED and buzzer OFF.
2. **Distance < 15 cm**: LCD shows Alert!!! (1st row) and distance (2nd row); LED blinks (2.5 s ON/OFF) and buzzer beeps only when LED is OFF.
3. **Otherwise**: LCD shows distance (1st row) and Approach: x% (2nd row); LED blinks (1.5 s ON/OFF) and buzzer beeps only when LED is OFF

Code:

```
#include <LiquidCrystal.h>

// LCD pin connections (RS, E, D4, D5, D6, D7)
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Pin definitions
#define TRIG_PIN 6
#define ECHO_PIN 8
#define LED_PIN 9
#define BUZZER_PIN 10

// Variables
long duration;
float distance;
int approachPercentage;

// Timing variables for non-blocking operation
unsigned long previousDistanceTime = 0;
unsigned long previousBlinkTime = 0;
const long distanceInterval = 500;  // Update distance every 500ms

// Blink state variables
bool ledOn = false;
bool blinkActive = false;
long currentBlinkInterval = 0;

void setup() {
  // Initialize LCD (16 columns, 2 rows)
  lcd.begin(16, 2);

  // Set pin modes
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // Turn off LED and buzzer initially
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  // Display startup message
  lcd.setCursor(0, 0);
  lcd.print("Initializing...");
```

```
  delay(2000);
  lcd.clear();
}

void loop() {
  unsigned long currentMillis = millis();

  // Measure and update distance at regular intervals
  if (currentMillis - previousDistanceTime >= distanceInterval) {
    previousDistanceTime = currentMillis;
    measureDistance();
    updateDisplay();
    setAlertState();
  }

  // Handle LED blinking and buzzer beeping if active
  if (blinkActive && currentBlinkInterval > 0) {
    if (currentMillis - previousBlinkTime >= currentBlinkInterval) {
      previousBlinkTime = currentMillis;
      toggleLEDAndBuzzer();
    }
  }
}

void measureDistance() {
  // Trigger ultrasonic sensor
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Read echo pulse
  duration = pulseIn(ECHO_PIN, HIGH, 30000); // Timeout after 30ms

  // Calculate distance in cm (speed of sound = 343 m/s = 0.0343 cm/µs)
  if (duration > 0) {
    distance = duration * 0.0343 / 2;
  }

  // Calculate approach percentage for middle zone
  approachPercentage = map(constrain(distance, 15, 45), 45, 15, 0, 100);
}

void updateDisplay() {
```

```
  lcd.clear();

  if (distance > 45) {
    // Case 1: Safe zone
    lcd.setCursor(0, 0);
    lcd.print("Safe");
    lcd.setCursor(0, 1);
    lcd.print("Dist: ");
    lcd.print(distance, 1);
    lcd.print(" cm");
  }
  else if (distance < 15) {
    // Case 2: Alert zone
    lcd.setCursor(0, 0);
    lcd.print("Alert!!!");
    lcd.setCursor(0, 1);
    lcd.print("Dist: ");
    lcd.print(distance, 1);
    lcd.print(" cm");
  }
  else {
    // Case 3: Warning zone
    lcd.setCursor(0, 0);
    lcd.print("Dist: ");
    lcd.print(distance, 1);
    lcd.print(" cm");
    lcd.setCursor(0, 1);
    lcd.print("Approach: ");
    lcd.print(approachPercentage);
    lcd.print("%");
  }
}

void setAlertState() {
  if (distance > 45) {
    // Safe zone - turn everything off
    blinkActive = false;
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
    ledOn = false;
  }
  else if (distance < 15) {
    // Alert zone - 2.5s blink interval
    blinkActive = true;
    currentBlinkInterval = 2500; // 2.5 seconds
```

```
  }
  else {
    // Warning zone - 1.5s blink interval
    blinkActive = true;
    currentBlinkInterval = 1500; // 1.5 seconds
  }
}

void toggleLEDAndBuzzer() {
  ledOn = !ledOn; // Toggle LED state

  if (ledOn) {
    // LED ON, Buzzer OFF
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, LOW);
  } else {
    // LED OFF, Buzzer ON (beeps)
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, HIGH);
  }
}
```

Output

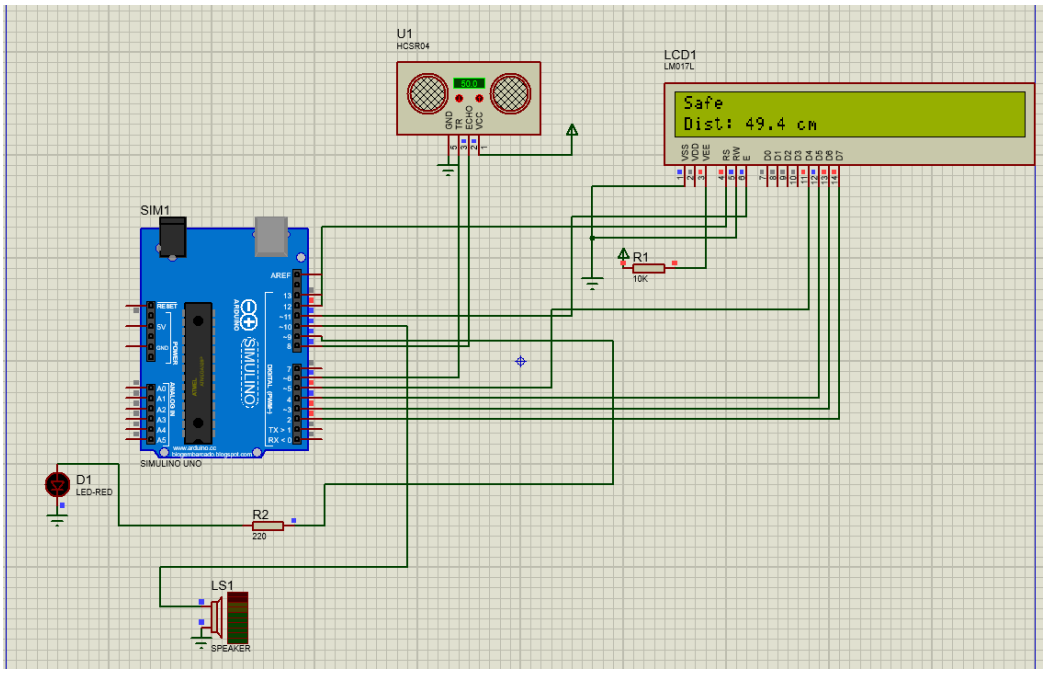


Figure 1: If distance is greater than 45 cm

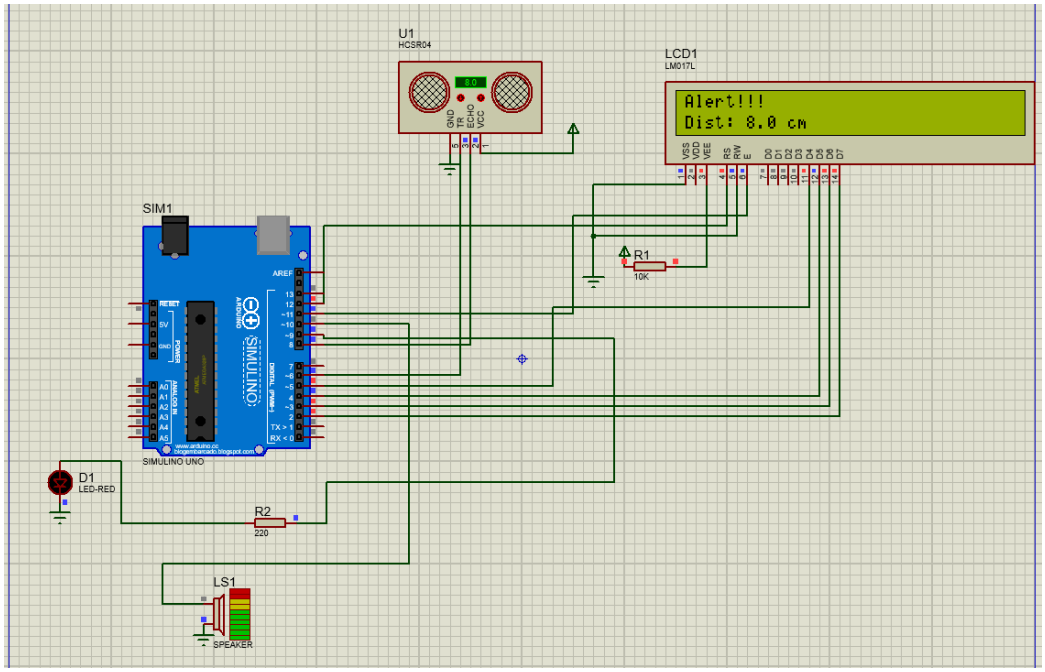


Figure 2: If distance is lower than 15 cm

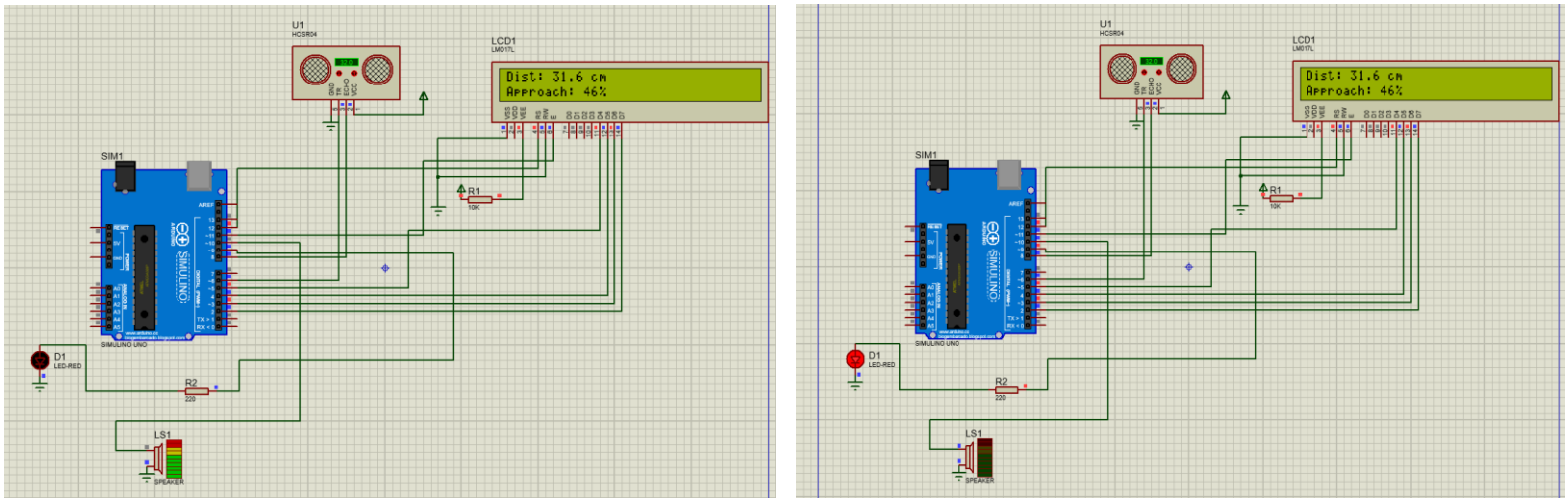


Figure-3: Led OFF & Speaker ON

Led ON & Speaker OFF